

Recorregut Hamiltonià amb Backtracking

Pràctica 2

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria presenta el desenvolupament d'una aplicació Java Swing que resol el problema del recorregut hamiltonià sobre un tauler d'escacs compartit per dues peces que alternen torns, amb la restricció que cap peça no pot estar en posició de capturar l'altra en cap moment del recorregut. L'algorisme emprat és *backtracking* recursiu amb la heurística de Warnsdorff per a l'ordenació de moviments. L'aplicació suporta sis tipus de peces —quatre clàssiques (cavall, reina, torre, alfil) i dues personalitzades (mussol i serp)— amb moviments estàtics i continus, taulers configurables de 4×4 fins a 12×12 , visualització pas a pas amb animació, predicció de temps d'execució i indicador de dificultat. L'arquitectura segueix el patró Model-Vista-Controlador (MVC) amb concurrència gestionada mitjançant *SwingWorker*.

Termes d'índex—recorregut hamiltonià, backtracking, heurística de Warnsdorff, Model-Vista-Controlador, Java Swing, *SwingWorker*, peces d'escacs, concurrència

1. Introducció

El problema del recorregut hamiltonià consisteix a trobar un camí que visiti exactament una vegada cada vèrtex d'un graf [1]. En el context d'un tauler d'escacs, cada casella és un vèrtex i les arestes connecten caselles segons les regles de moviment d'una peça determinada. El cas més conegut és el *Knight's Tour*, on un cavall ha de visitar totes les caselles d'un tauler 8×8 .

Aquesta pràctica generalitza el problema en dues direccions. Primera, en lloc d'una sola peça, el tauler és compartit per **dues peces que alternen torns**, de manera que cada peça visita aproximadament la meitat de les caselles. Segona, s'incorpora una **restricció de no-captura**: en cap pas del recorregut una peça pot estar en posició d'atacar l'altra. Aquesta restricció converteix el problema clàssic de recorregut hamiltonià en un problema de satisfacció de restriccions (*constraint satisfaction problem*, CSP) amb backtracking.

A més, l'aplicació suporta sis tipus de peces amb models de moviment fonamentalment diferents —moviment estàtic (salt fix) i moviment continu (lliscament fins a obstacle)—, la qual cosa genera una gran varietat de grafs implícits sobre el mateix tauler i fa que la dificultat variï dràsticament segons la combinació de peces.

L'objectiu de la pràctica és triple:

- 1) Implementar un algorisme de backtracking recursiu amb heurístiques d'optimització per resoldre el problema.
- 2) Construir una interfície gràfica amb Java Swing que permeti configurar els paràmetres, visualitzar

el procés de resolució i navegar la solució pas a pas.

- 3) Seguir estrictament el patró MVC amb concurrència gestionada per *SwingWorker*, garantint una separació neta entre la lògica de negoci i la interfície gràfica.

La memòria s'estructura de la següent manera: la Secció II descriu els fonaments teòrics; la Secció III detalla el disseny i la implementació; la Secció IV analitza l'algorisme de backtracking i les heurístiques emprades; la Secció V presenta els resultats; i la Secció VI tanca amb les conclusions.

2. Fonaments Teòrics

2.1. Recorregut Hamiltonià

Un **recorregut hamiltonià** (o camí hamiltonià) en un graf $G = (V, E)$ és una seqüència de vèrtexs $v_1, v_2, \dots, v_{|V|}$ tal que cada vèrtex apareix exactament una vegada i existeix una aresta $(v_i, v_{i+1}) \in E$ per a tot i [1]. Determinar l'existència d'un camí hamiltonià és un problema NP-complet en el cas general, la qual cosa implica que no es coneix cap algorisme que el resolgui en temps polinòmic per a tots els grafs.

En el cas particular d'un tauler d'escacs $N \times M$, el graf implícit ve determinat per les regles de moviment de la peça. Per al cavall sobre un tauler 8×8 , el graf té 64 vèrtexs i aproximadament 168 arestes, i existeixen solucions per a la majoria de configuracions. Tanmateix, la generalització a dues peces amb restriccions de captura redueix dràsticament l'espai de solucions vàlides.

2.2. Backtracking Recursiu

El *backtracking* és una tècnica de cerca exhaustiva que explora l'espai de solucions de forma sistemàtica, construint solucions candidates de forma incremental i descartant les branques que violen les restriccions del problema [2]. L'esquema general és:

- 1) **Cas base:** si la solució parcial és completa, retorna èxit.
- 2) **Generació de candidats:** per a la posició actual, es generen tots els moviments vàlids.
- 3) **Validació:** per a cada candidat, es comprova que satisfà les restriccions.
- 4) **Recursió:** s'aplica el moviment i es crida recursivament.
- 5) **Retrocés:** si la recursió falla, es desfà el moviment i es prova el següent candidat.

La complexitat en el pitjor cas és exponencial, ja que pot ser necessari explorar totes les permutacions possibles. Per a un tauler de $N \times M$ caselles amb dues peces, el nombre de permutacions potencials creix de forma factorial amb el nombre de caselles. Això fa imprescindible l'ús d'heurístiques de poda per reduir l'espai de cerca.

2.3. Heurística de Warnsdorff

La heurística de Warnsdorff [4], proposada originalment per H. C. Warnsdorff el 1823 per al problema del recorregut del cavall, consisteix a prioritzar els moviments cap a caselles amb **menor nombre de sortides vàlides** (*onward moves*). La intuïció és que les caselles amb poques sortides han de ser visitades aviat, ja que posteriorment serien inaccessibles i farien fracassar la solució.

Formalment, per a cada casella candidata c es calcula el grau de sortida:

$$d(c) = |\{c' \in V : (c, c') \in E \text{ i } c' \text{ no visitada}\}| \quad (1)$$

i es trien primer les caselles amb $d(c)$ mínim. Això transforma l'exploració cega del backtracking en una cerca guiada que, en la pràctica, redueix dràsticament el nombre de retrocessos necessaris.

En aquesta implementació, la heurística s'aplica mitjançant el mètode `getValidMovesSorted()` de la classe `Board`, que ordena la llista de moviments vàlids per grau de sortida creixent abans de passar-la al procés de backtracking.

2.4. Restricció de No-Captura

La restricció addicional d'aquesta pràctica és que, en cada pas del recorregut, cap peça no pot estar en posició de capturar l'altra. Formalment, si p_1 i p_2 són les posicions actuals de les dues peces, amb tipus t_1 i t_2 respectivament, la restricció exigeix:

$$\neg \text{captura}(p_1, t_1, p_2) \wedge \neg \text{captura}(p_2, t_2, p_1) \quad (2)$$

on $\text{captura}(p, t, q)$ indica que una peça de tipus t a la posició p pot atacar la posició q . Aquesta comprovació es fa **bidireccionalment** (ambdues peces es comproven com a atacants) i és la principal font de poda de l'espai de cerca.

2.5. Moviments Estàtics vs. Continus

Les peces d'escacs presenten dos models de moviment fonamentalment diferents:

Moviment estàtic: salta a una posició fixa determinada per un vector de desplaçament. Cada vector genera exactament un moviment candidat. Exemples: cavall, mussol, serp.

Moviment continu: es desplaça en una direcció fins a trobar la vora del tauler o una casella ocupada. Cada vector de direcció genera múltiples moviments candidats (un per cada casella lliure al llarg de la direcció). Exemples: reina, torre, alfil.

Aquesta distinció afecta tant la generació de moviments com la detecció de captura, ja que les peces amb moviment continu tenen un radi d'atac que depèn de la configuració actual del tauler.

2.6. Patró Model-Vista-Controlador

El patró MVC [3] separa l'aplicació en tres capes independents. En aquesta pràctica:

- **Model:** el tauler (`Board`), el solver (`BacktrackingSolver`), el camí hamiltonià (`HamiltonianPath`), les mètriques (`SolverMetrics`) i els tipus de peça (`PieceType`).
- **Vista:** el marc principal (`MainFrame`), el panell de configuració (`ConfigPanel`), el tauler visual (`BoardPanel`), els controls (`ControlPanel`) i les estadístiques (`StatsPanel`).
- **Controlador:** l'orquestrador (`SolverController`), la tasca de fons (`SolverTask`) i el predictor de temps (`TimePredictor`).

La concurrència es gestiona amb `SwingWorker` [6], que executa l'algorisme de backtracking en un fil de fons i publica actualitzacions de progrés al fil EDT (*Event Dispatch Thread*) de forma segura.

3. Disseny i Implementació

3.1. Capa del Model

3.1.1. PieceType (Enum). Defineix els sis tipus de peces com a constants d'un enum. Cada constant encapsula el nom en català, una abreviatura, el tipus de moviment (`STATIC` o `CONTINUOUS`), els vectors de moviment i un factor de complexitat per a la predicció de temps. La Taula 1 resumeix les característiques de cada peça.

Taula 1. TIPUS DE PECES, MOVIMENTS I COMPLEXITAT.

Peça	Nom	Tipus	Moviments	Complexitat
Cavall	KNIGHT	Estàtic	8	1.0
Reina	QUEEN	Continu	8 dir.	1.5
Torre	ROOK	Continu	4 dir.	0.7
Alfil	BISHOP	Continu	4 dir.	0.8
Mussol	OWL	Estàtic	4	1.2
Serp	SNAKE	Estàtic	16	1.4

El mussol es mou en creu a distància 2, saltant per sobre de caselles ocupades. La serp combina 8 moviments en L estesa ($\pm 3, \pm 1$) i ($\pm 1, \pm 3$) amb els 8 moviments de rei, donant-li una gran mobilitat en espais oberts però amb un alt factor de complexitat per a la cerca. El mètode `toString()` retorna el nom en català, de manera que els `JComboBox<PieceType>` mostren automàticament les etiquetes correctes sense cap codi addicional.

3.1.2. Position (Record). Registre immutable de Java 16 que representa una coordenada (r, c) al tauler. Proporciona mètodes per calcular la distància Manhattan i comprovar adyacència. Com a *record*, genera automàticament `equals()`, `hashCode()` i `toString()`, garantint l'ús segur com a clau en mapes i conjunts.

3.1.3. Board. Classe central que representa el tauler $N \times M$. Manté una matriu `boolean[][]` de caselles visitades i un comptador `visitedCount` per detectar eficientment el cas base (totes les caselles visitades).

Els mètodes principals són:

- `getValidMoves(Position, PieceType)`: genera la llista de moviments vàlids. Per a peces estàtiques, aplica cada vector una vegada. Per a peces contínues, estén cada direcció fins a trobar la vora o una casella visitada.
- `getValidMovesSorted(Position, PieceType)`: ordena els moviments per la heurística de Warnsdorff (grau de sortida creixent), tal com s'expressa a l'equació (1).
- `canCapture(Position, PieceType, Position)`: comprova si una peça pot atacar una posició. Per a peces contínues, verifica la línia de visió sense obstacles.

3.1.4. HamiltonianPath. Emmagatzema la seqüència ordenada de posicions visitades i l'índex de la peça que ha fet cada moviment (0 o 1). Actua com una pila durant el backtracking (`addMove/removeLastMove`) i com a objecte de transferència de dades cap a la vista un cop trobada la solució.

3.1.5. SolverMetrics. Recull estadístiques en temps real: nombre d'iteracions, nombre de retrocessos, profunditat màxima de recursió i temps transcorregut. El temps es mesura amb `System.currentTimeMillis()` i es pot consultar tant durant l'execució (per a actualitzacions en viu) com després.

3.1.6. BacktrackingSolver. Implementa l'algorisme de backtracking recursiu. El mètode públic `solve()` inicialitza l'estat i crida el mètode privat `backtrack()`, que segueix l'esquema del Llistat 1.

```

1 boolean backtrack(curPos, curPiece,
2                 curIdx, otherPos, otherPiece,
3                 otherIdx, depth) {
4     if (board.visitedCount == totalCells)
5         return true; // Cas base
6     if (stopped) return false;
7     metrics.incrementIterations();
8     for (Position next :
9         board.getValidMovesSorted(
10            curPos, curPiece)) {
11         if (canCapture(next, curPiece,
12            otherPos, otherPiece))
13             continue; // Poda: captura directa
14         if (canCapture(otherPos, otherPiece,
15            next, curPiece))
16             continue; // Poda: captura inversa
17         board.setVisited(next);
18         solution.addMove(next, curIdx);
19         // Alternar torn
20         if (backtrack(otherPos, otherPiece,
21            otherIdx, next, curPiece,
22            curIdx, depth+1))
23             return true;
24         board.unsetVisited(next);
25         solution.removeLastMove();
26         metrics.incrementBacktracks();
27     }
28     return false;
29 }

```

Listing 1. Esquema del mètode backtrack.

Aspectes clau de la implementació:

- **Alternança de tornos:** a cada nivell de recursió, els paràmetres “actual” i “altre” s'intercanvien, de manera que les dues peces alternen moviments.
- **Doble comprovació de captura:** es verifica tant que la peça que es mou no ataquí l'altra com que l'altra no ataquí la nova posició, segons l'equació (2).
- **Cancel·lació cooperativa:** el camp `volatile boolean stopped` es comprova a cada iteració, permetent aturar la cerca de forma neta des del fil principal.
- **Actualitzacions de progrés:** cada 1 000 iteracions, s'invoca un *callback* per actualitzar les estadístiques a la interfície.

3.2. Capa del Controlador

3.2.1. SolverController. Orquestra tot el cicle de vida de la resolució. Les responsabilitats principals són:

- 1) **Inici:** llegeix la configuració del `ConfigPanel`, valida les posicions inicials, crea els objectes del model (`Board`, `BacktrackingSolver`) i llança un `SolverTask` en un fil de fons.
- 2) **Aturada:** estableix el *flag* de cancel·lació al solver mitjançant `SolverTask.stopSolver()`.
- 3) **Reinici:** atura qualsevol execució en curs, esborra el tauler i restableix l'estat visual.

- 4) **Predicció:** actualitza la predicció de temps i el nivell de dificultat cada vegada que la configuració canvia.
- 5) **Completat:** gestiona els tres resultats possibles: solució trobada, solució inexistent, o execució aturada per l'usuari.

3.2.2. SolverTask (SwingWorker). Encapsula l'execució del solver en un `SwingWorker<Boolean, Void>`. El mètode `doInBackground()` executa `solver.solve()` en un fil de fons. Les actualitzacions de progrés arriben al fil EDT mitjançant `SwingUtilities.invokeLater()`, i el mètode `done()` gestiona la finalització amb tractament d'excepcions (`CancellationException`, `InterruptedException`).

3.2.3. TimePredictor. Classe utilitària que estima el temps d'execució abans d'iniciar la cerca, basant-se en un model exponencial empíric:

$$\hat{T}(n) = k \cdot e^{\alpha \cdot n} \cdot \bar{c}_{peça} \quad (3)$$

on $n = \text{files} \times \text{columnes}$, $k = 0,001$ s és la constant base, $\alpha = 0,35$ és el factor de creixement, i $\bar{c}_{peça}$ és la mitjana dels factors de complexitat de les dues peces. El resultat es presenta en català amb cinc nivells de dificultat codificats per color, com mostra la Taula 2.

Taula 2. NIVELLS DE DIFICULTAT I LLINDARS DE TEMPS.

Nivell	Llindar	Color
Molt fàcil	< 1 s	Verd
Fàcil	1–10 s	Verd-groc
Moderat	10–60 s	Taronja-groc
Difícil	1–5 min	Taronja fosc
Molt difícil	> 5 min	Vermell

3.3. Capa de la Vista

3.3.1. MainFrame. `JFrame` principal que compon els quatre sub-panells en un `BorderLayout`: `ConfigPanel` a l'oest dins un `JScrollPane`, `BoardPanel` al centre, i `ControlPanel` amb `StatsPanel` al sud. Gestiona internament la navegació pas a pas (botons « i ») i l'animació amb `javax.swing.Timer`, i actua com a façana de delegació perquè el controlador interaccioni amb la vista a través d'un únic punt d'entrada.

3.3.2. ConfigPanel. Panell de configuració amb `GridBagLayout` que implementa la interfície `Scrollable` per permetre desplaçament vertical. Inclou:

- *Spinners* per a les dimensions del tauler (4–12 files i columnes).
- Selectors de tipus de peça (`JComboBox<PieceType>`) amb previsualització d'icona.

- *Spinners* per a les posicions inicials, amb validació de límits automàtica en canviar les dimensions.
- Botons de col·locació interactiva (`JToggleButton` en grup mutuament exclouent): l'usuari activa el mode de col·locació i fa clic directament sobre una casella del tauler per establir la posició inicial d'una peça.
- Etiquetes de predicció de temps i nivell de dificultat, actualitzades en temps real cada vegada que la configuració canvia.

3.3.3. BoardPanel. Panell que renderitza el tauler com una quadrícula de cel·les amb patró d'escacs (clar/fosc). La quadrícula interna es manté sempre **perfectament quadrada**, centrada dins l'espai disponible mittançant un *layout* nul amb càlcul manual de posició (la dimensió del costat és `min(ample, alt)`).

Cada cel·la dibuixa:

- 1) El fons amb colors que distingeixen cel·les visitades (verd clar/fosc) de les no visitades (blanc/marró).
- 2) El nombre de seqüència a la cantonada superior esquerra.
- 3) La icona de la peça (imatge PNG o text com a alternativa) centrada a la cel·la.
- 4) Un marc daurat per destacar el pas actual durant la navegació.

Les cel·les responen a clics del ratolí per a la col·locació interactiva de peces. El panell ofereix un mode de previsualització que mostra les posicions inicials de les dues peces sobre el tauler buit, amb l'icona real de cada peça carregada des dels recursos del projecte.

3.3.4. ControlPanel. Panell de controls amb dues files: botons d'acció a la fila superior i un control lliscant de velocitat d'animació a la fila inferior. Els botons són:

- Iniciar, Aturar, Reiniciar: gestió del cicle de vida del solver.
- «, »: navegació pas a pas per la solució.
- Reproduir/Pausar: animació automàtica de la solució.

El control lliscant ajusta el retard de l'animació entre 50 ms (ràpid) i 2000 ms (lent), amb actualització en temps real durant la reproducció. Tots els botons empen icones textuais amb caràcters Unicode del bloc de formes geomètriques (per exemple, ► per a reproduir i ■ per a aturar), evitant dependències d'imatges externes per als controls.

3.3.5. StatsPanel. Panell d'estadístiques que mostra en temps real: temps transcorregut (format MM:SS o HH:MM:SS), nombre d'iteracions, nombre de retrocessos (*backtracks*), profunditat màxima i estat del solver amb codificació de color. Inclou un `Timer` de `Swing` que actualitza el temps transcorregut cada segon durant l'execució.

4. Anàlisi de l'Algorisme

4.1. Complexitat Temporal

L'algorisme de backtracking explora un arbre de cerca on cada node correspon a una assignació parcial de posicions. En el pitjor cas, l'arbre té profunditat $N \cdot M$ i cada node pot tenir fins a b branques, on b depèn del tipus de peça i de la mida del tauler.

Per a un cavall en un tauler $N \times M$, el grau màxim és 8. Per a una reina, el grau pot arribar a $2(N + M) - 2$. La complexitat en el pitjor cas és:

$$T_{\text{pitjor}} = O(b^{N \cdot M}) \quad (4)$$

Tanmateix, la heurística de Warnsdorff redueix dràsticament el nombre de nodes explorats a la pràctica. Per al cavall sobre un tauler 8×8 , és habitual trobar la solució en menys de 64 passos (sense retrocessos) quan la heurística és efectiva.

4.2. Complexitat Espacial

L'espai addicional consumit per l'algorisme és:

$$S = O(N \cdot M) \quad (5)$$

corresponent a la matriu de caselles visitades i la llista de posicions del camí. La profunditat de la pila de recursió és com a màxim $N \cdot M$.

4.3. Impacte de les Restriccions de Captura

La restricció de no-captura actua com un filtre sobre els moviments candidats. L'impacte varia segons el tipus de peça:

- **Peces estàtiques** (cavall, mussol, serp): la comprovació de captura és $O(1)$ per moviment, ja que cal comprovar si la posició de l'altra peça coincideix amb un dels desplaçaments fixos.
- **Peces contínues** (reina, torre, alfil): la comprovació requereix verificar la línia de visió, que és $O(\max(N, M))$ per moviment en el pitjor cas.

4.4. Predicció de Temps

El model de predicció emprava una funció exponencial (equació 3) calibrada empíricament. La justificació del model exponencial és que el nombre de nodes de l'arbre de cerca creix exponencialment amb el nombre de caselles. Els factors de complexitat per peça reflecteixen la diferència observada en temps d'execució entre combinacions de peces: la reina ($c = 1,5$) genera molts més retrocessos que la torre ($c = 0,7$).

5. Resultats i Discussió

L'aplicació implementada permet resoldre el problema del recorregut hamiltonià per a una àmplia varietat de configuracions. Les observacions principals són:

- **Taulers petits** (4×4 , 5×5): les solucions es troben en menys d'un segon per a la majoria de combinacions i posicions inicials.
- **Taulers mitjans** (6×6 , 7×7): el temps varia significativament segons el tipus de peça. Dues torres troben solucions ràpidament; dues reines poden requerir minuts.
- **Taulers grans** (8×8 i superiors): els temps creixen de forma exponencial i algunes configuracions poden no trobar solució en temps raonable.

La heurística de Warnsdorff és essencial per a la viabilitat pràctica de l'algorisme. Sense ella, fins i tot taulers de 5×5 poden requerir milions de retrocessos.

La funcionalitat de navegació pas a pas i l'animació amb velocitat ajustable permeten a l'usuari comprendre visualment com les dues peces cobreixen el tauler i verificar que la solució és correcta.

La col·locació interactiva de peces facilita l'experimentació amb diferents posicions inicials i permet observar com la posició de partida influeix dràsticament en el temps de resolució.

Respecte al disseny, l'arquitectura MVC ha facilitat la separació entre l'algorisme (model) i la visualització (vista), la gestió de la concurrència al controlador, i la incorporació de funcionalitats voluntàries sense modificar el model.

6. Conclusions

Aquesta pràctica ha permès implementar i analitzar un algorisme de backtracking recursiu per al problema del recorregut hamiltonià amb dues peces, aplicant la heurística de Warnsdorff i restriccions de no-captura.

Des del punt de vista algorísmic, s'han verificat experimentalment els aspectes fonamentals del backtracking: el creixement exponencial del temps d'execució amb la mida del tauler, l'efecte dramàtic de les heurístiques d'ordenació de moviments, i la importància de la poda per restriccions.

Des del punt de vista del disseny, l'aplicació demostra una implementació MVC on:

- 1) El model és completament independent de la interfície gràfica i es podria reutilitzar en un context diferent.
- 2) La concurrència es gestiona amb `SwingWorker`, garantint que la interfície roman responsiva durant cerques que poden durar minuts.
- 3) La cancel·lació cooperativa mitjançant un camp `volatile` permet aturar la cerca de forma neta.
- 4) Les funcionalitats de visualització (navegació pas a pas, animació, previsualització de peces, col·locació interactiva) enriqueixen l'experiència d'usuari sense afectar la lògica del model.

Com a treball futur, es podrien explorar heurístiques addicionals (detecció de components desconnectats, cerca bidireccional), implementar la cerca en paral·lel, o calibrar el model de predicció amb dades experimentals per millorar la seva precisió.

Referències

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.
- [3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [4] H. C. Warnsdorff, *Des Rösselsprunges einfachste und allgemeinste Lösung*. Schmalkalden, Germany, 1823.
- [5] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.
- [6] Oracle Corporation, “The Java Tutorials — Concurrency,” [Online]. Available: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- [7] J. Elliott, R. Eckstein, M. Loy, D. Wood, and B. Cole, *Java Swing*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2002.